

CODEFUSION: PYTHON & JAVA SHOWDOWN

PYTHON & JAVA-BASED CODING COMPETITION

ROUND 3 - Console-Based Application Development (Python/Java)

Objective:

Participants must develop a fully functional console-based application using either Python or Java within 2 hours.

Rules:

1. **Total Time: 2 Hours**
2. Participants must use Python or Java only.
3. Choose One Problem Statement
4. Internet access is restricted – No external help.
5. Submissions must be completed within the given time.
6. Code must be functional and error-free.
7. Use of external libraries is not allowed (except built-in ones).
8. Proper indentation, comments, and readability are mandatory.
9. Evaluation by Judges Based on Functionality & Optimization

Submission Guidelines

- Participants must submit one final working file before the deadline.
- The code must compile and execute successfully.
- The application must meet the minimum functionality requirements.
- Bonus points will be given for exception handling, user-friendly prompts, and extra features.

Problem Set for Console-Based Applications

Problem 1: Library Management System

Scenario:

Develop a **Library Management System** that allows users to:

- **Add books**
- **Borrow books** (Check availability)
- **Return books**
- **Display available books**

Features:

- Maintain a list of books with their availability status.
- Allow users to borrow and return books.
- Display all available books in the library.
- Prevent borrowing a book that is already borrowed.

Extra Features for Bonus Points:

- Implement **search by author, genre, or book title**.
- Add a **due date system** for borrowed books and **late fee calculation**.
- Introduce a **user authentication system** (Admin can add/remove books, Users can borrow).

- Store **borrowed book history** for each user.
- Implement **exporting book catalog and user records to a file**.

Problem 2: ATM Simulation System

Scenario:

Develop an **ATM simulation system** that allows users to:

- **Check balance**
- **Withdraw money**
- **Deposit money**
- **Exit**

Features:

- Users should have an initial balance.
- Allow withdrawals only if sufficient balance is available.
- Allow deposits to increase balance.
- Show error messages for incorrect transactions.

Extra Features for Bonus Points:

- Implement **multi-user login authentication (PIN-based login)**.
- Track **transaction history (Deposits, Withdrawals, Balance Inquiries)**.
- Add **fund transfer functionality** between accounts.
- Introduce a **daily withdrawal limit** per user.
- Implement **fake transaction alerts (for suspicious activities)**.

Problem 3: Student Management System

Scenario:

Develop a **Student Management System** that allows:

- **Adding student details (Name, Roll No, Marks)**
- **Viewing all students**
- **Searching for a student by roll number**
- **Deleting a student record**

Features:

- Store student details in a list (Python) or ArrayList (Java).
- Allow searching students by roll number.
- Allow deletion of a student record.
- Display a list of all students.

Extra Features for Bonus Points:

- Implement **search by name or roll number**.
- Add **grade calculation (A, B, C, etc., based on marks)**.
- Allow **import/export of student data from/to a file**.
- Introduce a **simple login system (Admin can modify records, Students can view only)**.

- Provide **ranking of students based on marks**.

Problem 4: Online Shopping Cart System

Scenario:

Develop a **Shopping Cart System** where users can:

- **View product catalog**
- **Add items to cart**
- **Remove items from cart**
- **Checkout and display total price**

Features:

- Maintain a list of available products with prices.
- Users can add multiple items to the cart.
- Show total price at checkout.
- Prevent adding non-existent products.

Extra Features for Bonus Points:

- Implement **user login and purchase history tracking**.
- Add **discount codes and promotional offers**.
- Introduce **inventory management (Stock levels for products)**.
- Allow **wishlist functionality** (Users can save products for later).
- Generate **receipt with itemized bill and tax calculations**.

Problem 5: Employee Payroll Management System

Scenario:

Develop an **Employee Payroll System** that allows:

- **Adding employees (Name, ID, Salary)**
- **Viewing payroll details**
- **Updating employee salary**
- **Deleting an employee record**

Features:

- Store employee details in a list (Python) or ArrayList (Java).
- Allow modifications to salary.
- Allow deletion of an employee record.
- Display all employees' payroll details.

Extra Features for Bonus Points:

- Implement **multi-user roles (HR can modify, Employees can view only)**.
- Add **salary slip generation and export to a file**.
- Introduce **tax deduction calculations based on salary slabs**.

- Allow **automatic yearly increments based on performance**.
 - Implement **bonus system for top-performing employees**.
-

Problem 6: Hotel Room Booking System

Scenario:

Develop a **Hotel Room Booking System** that allows users to:

- **View available rooms**
- **Book a room**
- **Cancel a booking**
- **Display current reservations**

Features:

- Maintain a list of rooms with their availability status.
- Allow users to book and cancel rooms.
- Prevent double booking of the same room.

Extra Features for Bonus Points:

- Implement **customer details storage** (name, contact number).
 - Allow **searching reservations by customer name**.
 - Provide **automatic room allocation based on availability**.
 - Generate a **simple invoice** for booked rooms.
-

Problem 7: Vehicle Parking System

Scenario:

Develop a **Parking System** that allows users to:

- **View available parking slots**
- **Park a vehicle**
- **Remove a vehicle**
- **Show parked vehicles**

Features:

- Maintain parking slots with occupied/free status.
- Prevent parking if slots are full.
- Allow users to remove vehicles by entering their number.

Extra Features for Bonus Points:

- Implement **charges per hour** and show total cost when a vehicle exits.
 - Maintain **parking history with entry and exit timestamps**.
 - Allow **pre-booking of parking slots**.
 - Generate **receipt with vehicle details and total cost**.
-

Problem 8: Flight Reservation System

Scenario:

Develop a **Flight Reservation System** that allows:

- **Viewing available flights**
- **Booking a ticket**
- **Cancelling a ticket**
- **Showing all reservations**

Features:

- Maintain a list of flights with available seats.
- Prevent booking if no seats are available.
- Allow users to cancel a reservation.

Extra Features for Bonus Points:

- Implement **seat selection** for economy/business class.
 - Add **frequent flyer discounts** for repeat customers.
 - Include **meal preferences selection**.
 - Generate **electronic boarding passes** with passenger details.
-

Problem 9: Expense Tracker Application

Scenario:

Develop an **Expense Tracker** that allows users to:

- **Add an expense**
- **View all expenses**
- **Delete an expense**
- **View total expenses**

Features:

- Users can add expenses with a description and amount.
- The application stores all expenses and shows total spending.
- Users can delete an expense by selecting an entry.

Extra Features for Bonus Points:

- Implement **category-based tracking (Groceries, Travel, Food, etc.)**.
 - Provide **monthly budgeting alerts** when expenses exceed a limit.
 - Allow **exporting expense reports to a text file**.
 - Show **highest and lowest spending categories**.
-

Problem 10: Quiz Management System

Scenario:

Develop a **Quiz Management System** that allows:

- **Creating a quiz**
- **Adding questions to a quiz**
- **Taking a quiz**
- **Viewing results**

Features:

- The system should allow multiple-choice questions.
- Users should answer questions and receive a score.
- Display final results after completing the quiz.

Extra Features for Bonus Points:

- Implement **timed quiz mode** with a countdown timer.
- Include **multiple difficulty levels (Easy, Medium, Hard)**.
- Allow users to **review incorrect answers after the quiz**.
- Generate **leaderboards for high scores**.